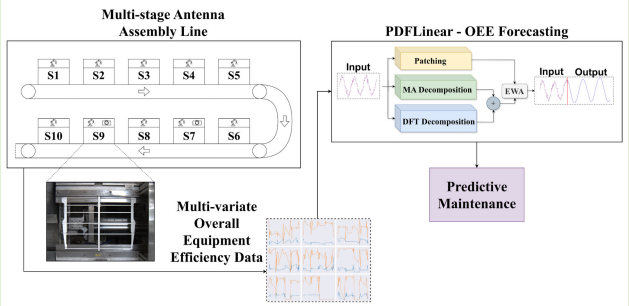
Time-Series Forecasting in Industrial Environments: A Performance Study and a Novel Late Fusion Framework

Dimitrios Oikonomou, Lampros Leontaris, *Member, IEEE*, Nikolaos Dimitriou, *Member, IEEE*, and Dimitrios Tzovaras, *Senior Member, IEEE*

***Abstract*—In manufacturing environments, monitoring of the overall equipment effectiveness (OEE) via soft sensors plays a pivotal role in enhancing productivity and efficiently planning maintenance schedules. However, the accurate forecasting of the OEE presents considerable challenges due to the complexity of manufacturing data and equipment interdependence across stages. To this end, advanced time-series forecasting methods based on deep learning (DL) pose a promising avenue in tackling these challenges. In this study, we present a taxonomy of DL forecasting architectures, consisting of multilayer perceptrons (MLPs), recurrent models, Transformer-based models, and temporal convolutional networks (TCNs), and we perform a comparative study of the state-of-the-art approaches. Additionally, a lightweight late fusion linear architecture is proposed, incorporating patching, moving average (MA) decomposition, and Fourier Transform decomposition (PDFLinear), and an exponentially weighted averaging (EWA) module responsible for late fusion. Representative state-of-the-art models of each taxonomy class are benchmarked using a real-world antenna assembly line use case and compared against our proposed method. The experimental results show that our proposed model consistently matches or outperforms the state-of-the-art models in terms of forecasting efficacy for all forecast horizons, while requiring a fraction of the computational resources.**



***Index Terms*— Deep learning (DL), hybrid models, predictive maintenance, sensor data, time-series forecasting.**

I. Introduction

In THE era of Industry 4.0, predictive analytics and monitoring in manufacturing settings have garnered significant attention for their crucial role in process optimization. As manufacturing systems become increasingly complex, the integration of AI-driven soft sensors has become essential for extracting insights from data that would otherwise be difficult or impossible to directly measure. Soft sensors are AI-driven virtual models that estimate unmeasured production variables. By leveraging historical data, soft sensors enable the forecasting of future production trends and the early detection of upcoming failures. The main tasks of soft sensors include: 1) automated equipment calibration and reconfiguration [1], [2]; 2) defect detection and prediction models [3], [4], [5]; 3) real-time process monitoring and control systems [6], [7]; and 4) predictive maintenance frameworks [8], the core focus of this study.

These objectives are integral to predictive maintenance strategies, aligning with the principles of the zero defect manufacturing (ZDM) framework [9], [10], [11], which emphasizes the minimization or elimination of defects in manufacturing processes through advanced monitoring, data-driven decision-making, and automation. The ZDM framework represents a paradigm shift toward holistic quality management, offering pathways to achieve sustainability and resilience in production systems.

AI-enabled sensors facilitate predictive maintenance by continuously assessing machine conditions, providing timely predictions for potential failures, and allowing for early intervention that minimizes equipment downtime and improves operational efficiency. At the core of predictive analytics lies time-series forecasting, a statistical tool that enables the forecasting of future states of manufacturing equipment based on historical data. Accurate forecasting facilitates informed

Received 26 November 2024; revised 23 December 2024; accepted 23 December 2024. Date of publication 13 January 2025; date of current version 13 February 2025. This work was supported by the European Commission, in part through Project OPTIMAI funded by the European Union's Horizon 2020 Frame-work Programme under Grant 958264, and in part through Project INDUX-R funded by the European Union's Horizon Europe (HORIZON) programme under Grant 101135556. The associate editor coordinating the review of this article and approving it for publication was Dr. Andre Eugenio Lazzaretti. *(Corresponding author: Dimitrios Oikonomou.)*

The authors are with the Information Technologies Institute, Centre for Research and Technology Hellas (CERTH-ITI), 57001 Thessaloniki, Greece (e-mail: doikonomou@iti.gr; lleontar@iti.gr; nikdim@iti.gr; dimitrios.tzovaras@iti.gr).

Digital Object Identifier 10.1109/JSEN.2025.3526362

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

and proactive decision-making, leading to reduced downtime, optimized maintenance schedules, and improved OEE. Furthermore, predictive maintenance enables sustainable manufacturing practices by improving system reliability, reducing waste, and minimizing resource consumption, aligning maintenance strategies with both operational and environmental objectives [12].

In this study, we focus on a highly specialized manufacturing process, an antenna assembly line, which serves as a test case for applying advanced DL forecasting methods. The antenna assembly line is a highly complex process with temporal and spatial interdependencies, which continuously generates a large amount of data. These data encompass a wide range of metrics, including production volumes, defect rates, manufacturing cycle times, and equipment downtimes. These metrics are integral to the calculation of OEE, a critical performance indicator that reflects the efficiency and effectiveness of manufacturing operations. The ability to forecast OEE and its constituent factors with high accuracy is essential for ensuring seamless production, minimizing defects through timely equipment maintenance, and ultimately driving productivity gains [13].

Traditional statistical forecasting methods, such as autoregressive integrated moving average (ARIMA) [14] and exponential smoothing (ES) [15], have been widely used in time-series forecasting, in various applications including finance and healthcare, for their simplicity and interpretability. However, these methods often struggle with capturing nonlinear patterns, long-range dependencies, and the intricate dynamics of modern manufacturing environments. In response to these challenges, this study explores advanced deep learning (DL) techniques, which have shown promise in capturing complex, nonlinear relationships in time-series data, and applies them to OEE forecasting in an antenna assembly line use-case. Although the primary focus of our study is on modeling multistage processes with complex cross-dependencies, as exemplified by the antenna assembly line dataset, we also examine a single-stage process from the sink manufacturing industry. This addition enables us to assess the generalizability and adaptability of our proposed method across diverse manufacturing scenarios.

We categorize DL approaches to time-series forecasting into four model architecture classes: 1) multilayer-perceptrons (MLPs); 2) recurrent neural networks (RNNs); 3) transformer or attention-based; and 4) temporal convolutional networks (TCNs). Each category presents unique advantages and challenges.

1) MLPs, though not inherently suited for capturing temporal dependencies, can be adapted for time-series forecasting through specialized feature engineering and data transformation techniques [16], [17].
2) RNNs (e.g., RNNs, LSTMs, GRUs) are widely used in time-series forecasting for their ability to capture temporal patterns [18], [19], [20], but they may struggle with vanishing gradients and limited parallelization in complex or long sequences [21].
3) Transformer-based models, adapted from NLP, excel at handling long-range dependencies and parallelizing computation for time-series forecasting [22], [23], [24], though their high computational demands can be a limitation in resource-constrained settings [25].
4) TCNs efficiently model complex temporal patterns using convolutional layers, but their success is highly dependent on hyperparameter tuning and deep networks to handle long input lengths and forecast horizons [26].

This article introduces a lightweight MLP-based model that integrates multiple feature extraction techniques, including: 1) PatchTST's [27] patching mechanism for efficient short-range feature extraction, a moving average (MA) [16], [17] seasonal decomposition scheme; and 2) a discrete Fourier transform (DFT) [28] decomposition scheme for mid-to-long-range forecast performance. Additionally, a simple late-fusion module is developed that adaptively combines the forecasts of the three predictor blocks using an exponentially weighted averaging (EWA) mechanism. This approach leverages the strengths of each individual predictor feature extraction technique, resulting in a more robust and accurate forecast across varying forecast horizons.

The objective of this study is to address the challenges in time-series forecasting in a real-world manufacturing setting, provide valuable insights into the applicability and effectiveness of different forecasting techniques in a real-world manufacturing setting.

The main contributions of this study are as follows.

1) We identify and present a taxonomy of the state-of-the-art of DL forecasting methods, and perform a comparative analysis across various forecast horizons and evaluation metrics.
2) Propose a lightweight late-fusion forecasting method that integrates multiple temporal feature extraction techniques.
3) Apply the proposed method to a real-world antenna assembly line use-case, developing soft sensors for equipment efficiency forecasting and enabling predictive maintenance.

In the following sections, we will present the model architecture taxonomy and related work (Section II), outline the methodology proposed in this study (Section III), present the results of our model evaluations (Section IV), and discuss the implications of our findings for the field of time series forecasting in manufacturing (Section V).

II. Literature Review

A. DL Methods' Taxonomy

This section provides a comprehensive taxonomy of DL methods, categorized according to their architectural foundations. This taxonomy serves as a framework for identifying the strengths and applications of each method.

1) Multilayer Perceptrons: MLPs comprise the most fundamental and widely used neural network architecture, consisting of a single or multiple layers of neurons, where the output of each layer serves as the input for the subsequent layer. A characteristic example of such an architecture is N-BEATS [29], a neural architecture that does not rely on temporal feature engineering and consists solely of MLP-based blocks that are organized hierarchically into stacks with doubly residual connections.

Whereas N-BEATS provides a generic sequence prediction solution with no reliance on specialized time-series modules, in recent years, time-series-specific neural architectures have made great advances. By incorporating temporal feature extraction mechanisms into their base architecture, they enhance the model's ability to capture temporal patterns, while also reducing random noise present in the data that may lead to overfitting and reduction of performance. Such architectures include N-HiTS [30], which extends N-BEATS with the inclusion of downsampling and max pooling layers within the base block architecture to reduce model complexity and data noise, and DLinear [16], which leverages an MA decomposition scheme combined with a simple MLP model. Additionally, FreTS [31], which leverages Fourier transform (FT) decomposition to process the input time series on its frequency-domain representation, capturing long-term dependencies, and Koopa [32], which leverages an encoder–decoder architecture to learn Koopman embeddings of the input time-series and decode it into forecasts.

Furthermore, several ensemble architectures have been proposed that fuse the outputs of simpler base models, leveraging the strengths of each constituent model, mitigating individual weaknesses, and ultimately improving the forecasting performance. Such examples include LightTS [33], which utilizes continuous and interval sampling to train two constituent models, whose outputs are fused, generating the final forecast. TimeMixer [17], on the other hand, utilizes multiscale downsampling and MA decomposition to train individual MLP predictor models, whose outputs are then fused using a mixing layer. It outperforms state-of-the-art Transformer and TCN models across multiple domains and benchmark datasets.

2) Recurrent Neural Networks: RNNs represent a class of neural network architectures where the output of a model at every step is fed back into the model as input for the next step, allowing the network to process sequences of data, such as time-series, and capture temporal dependencies over time. The most common architecture of recurrent models is that of RNNs, a deep neural network architecture that computes recursively both a prediction and a hidden state representation. For each time step, the current time-step input along with previous time-step hidden state are used to forecast the next time-step value, allowing the model to retain memory of previous values and model temporal dependencies [34].

Although RNNs have exhibited strong performance in forecasting tasks over the years, they face certain challenges, such as the "vanishing gradient" problem, which hinders their ability to adequately model long-range dependencies in long-term forecasting [21]. Several RNN variants have been introduced in order to tackle this problem, including long short-term memory (LSTM) networks [35], which introduce a complex memory cell structure that can maintain important long-range information, and gated recurrent units (GRUS) [36], which simplify LSTMs, increasing their computational efficiency and reducing overfitting. Additionally, all three RNN variants can be configured in a bidirectional fashion (bi-LSTM, bi-GRU), where the computation of hidden states and predictions at each time step incorporates both past and future information, enhancing predictive accuracy of the models [37].

Echo state networks (ESNs) represent a type of RNN designed to address the challenges of training traditional RNNs, such as the vanishing gradient problem, by organizing randomly connected neurons into a "reservoir" that projects the input time-series into high-dimensional space representations and simplifying the training process by only adjusting the output weights [38]. Quasi RNNs (QRNNs) represent a model architecture that leverages the base recursive structure of RNNs and combines it with the strengths of convolutional neural networks (CNNs) by alternating convolutional layers, which process the input sequences in parallel, and pooling layers, which are inspired by the gate mechanisms of LSTMs [39]. Extended long short-term memory (xLSTM) is a recent novel RNN architecture designed to tackle the limitations of LSTMs by replacing the sigmoid functions in their gating mechanisms with exponential functions that allow for more adaptive control over the flow of information, and introducing more complex memory cell structures that enhance the modeling capability and the memory performance over long ranges [40].

Legendre memory units (LMUs) comprise yet another extension of LSTMs, in this case through the use of Legendre polynomials to parametrize the memory component of the network. This allows the model to efficiently maintain information across longer data sequences, while requiring fewer computational resources [41], [42]. FiLM combines the ideas behind LMUs with FT decomposition to reduce random noise in the data, yielding improved performance over other state-of-the-art models in multivariate and univariate forecasting by 20.3% and 22.6%, respectively [43].

Finally, state space models (SSMs) comprise a mathematical framework used to model dynamic systems by representing the system's internal states and their evolution over time through state transition and observation equations [44]. In recent years, the ideas behind SSMs have been successfully applied to build deep recursive time-series forecasting models, including the linear spate-space layer (LSSL) [45], which combines the strengths of RNNs, CNNs, and continuous-time models. The structured state-space sequence model (S4) [46] and the simplified state-space sequence model (S5) [47] tackle certain computational challenges of the LSSL. Mamba [48], an advanced neural network architecture designed to efficiently handle long sequences by combining memory augmentation with model-based attention mechanisms, enhances the model's ability to capture and utilize long-term dependencies in data.

3) Transformers: The recent groundbreaking work on Transformers and attention-based models has revolutionized natural language processing by using a self-attention mechanism [49]. This enables parallelization and improves the model's ability to capture long-range dependencies in sequences, outperforming previous models like RNNs and LSTMs in various time-series forecasting tasks [50]. Nevertheless, the canonical Transformer architecture faces various challenges regarding time-series forecasting, which include: 1) requiring large amounts of data for training; 2) high computational costs due to quadratic complexity at $O(L^2)$; 3) difficulty in capturing temporal dependencies; and 4) overfitting, especially on highly noisy data.

In order to tackle the complexity problem and reduce the computational costs of the canonical Transformer architecture, Reformer leverages locality-sensitive hashing (LSH) for efficient attention computation and reversible residual layers [51], whereas Informer uses a probabilistic sparse self-attention mechanism and distilling operations [52]. Additionally, several time-series decomposition schemes have been integrated into the attention mechanism, including the MA decomposition of autoformer [53], the DFT decomposition of FEDformer [28], and the ES decomposition of ETSformer [54]. Other recent attention-based models include PatchTST [27], which divides the input time-series into discrete patches and processes them individually, achieving an overall reduction of 21.0% on MSE compared to other Transformer-based models, and iTransformer [55], which inverts the dimensions of the input time series, reducing complexity, achieving a decrease of 38.9% on MSE over the canonical Transformer and outperforming PatchTST on six out of seven benchmark datasets.

4) Temporal Convolutional Networks: TCNs are a class of deep neural networks specifically designed for sequence modeling tasks, where they use causal convolutions and dilation to capture long-range dependencies without the risk of information leakage from future to past in the input sequence. TCNs combine the advantages of CNNs (such as parallelism and stable gradients) with the ability to handle temporal data efficiently, making them well-suited for tasks like time-series forecasting and sequential data analysis [56]. Nevertheless, training TCNs face several challenges, including achieving a long effective history size, which requires an extremely deep network or very large filters which increase the computational complexity of the model. Additionally, TCNs require careful parameter adjustment and fine-tuning, especially when adapting to new datasets or application domains with differing temporal characteristics. This is particularly crucial in tasks where the length and nature of historical dependencies vary, as improper tuning can lead to suboptimal model performance, either by failing to capture necessary long-range dependencies or by overfitting to noise in the data [26].

Examples of such model architectures include the multiscale isometric convolution network (MICN) [57] and TimesNet. MICN leverages downsampling convolutions and isometric convolution to capture temporal features at different resolutions, while maintaining geometric isometry. It achieves an average MSE reduction of 17.2% and 21.6%, in multivariate and univariate experiments, respectively, across six benchmark datasets. TimesNet [58] utilizes fast Fourier transform (FFT) to transform the input time-series into period-based 2-D representations and then processes them using 2-D CNN backbone models, including popular vision models such as ResNet [59], ResNeXt [60] and ConvNeXt [61]. It achieves state-of-the-art or better results in 80% of time-series forecasting tasks.

State-of-the-art reviews in time-series forecasting generally fall into two distinct categories, each with its own limitations. The first category encompasses case studies where models are evaluated in real-world applications across various domains, such as in finance [62], weather [63], healthcare [64], or industry [65]. However, these studies often rely on established statistical models (e.g., ARIMA, Bayesian, SVM, etc.), or basic variants of DL models—such as simple MLPs, CNNs, or LSTMs. The second category includes studies that provide a comprehensive review of recent model architectures. Even though these studies consider state-of-the-art advancements, the models are typically tested only on benchmark datasets, which may be either real-world [58], [66], [67] or synthetic [68]. In our work, we expand the research on time-series forecasting and bridge these gaps by evaluating a wide range of advanced models on a real-world dataset with practical applications.

B. Process Optimization in Industrial Applications

As discussed in Section I, process optimization has become a crucial component in modern manufacturing and industrial applications in the transition toward Industry 4.0. This section reviews recent advancements and approaches in process optimization leveraging virtual metrology, soft sensors, and time-series forecasting, integrating both traditional and AI-driven approaches across various industrial sectors. Virtual metrology and soft sensors represent pivotal technologies within the Industry 4.0 ecosystem and the ZDM framework. These tools enable real-time estimation of product quality and process parameters without relying on direct physical measurements, significantly enhancing efficiency and reducing costs [69].

Multiple studies have shown the efficacy of RNNs in predictive maintenance, including a study by Sagheer and Kotb [19], who developed a deep LSTM model for forecasting petroleum production, whereas Chen et al. [20] designed a DL ensemble model, fusing a bi-LSTM with an MLP-based autoencoder to forecast the remaining useful life (RUL) of turbofan engines. In another approach for RUL prediction by Gao et al. [70], a hybrid framework was developed, combining a dual channel feature attention module for feature extraction with domain-specific feature predictors, including a CNN predictor for space-domain and a bi-LSTM predictor for time-domain features. The efficacy of hybrid CNN-LSTM systems has also been verified by the work of Essien and Giannetti [71] in an aluminum can manufacturing application. Several ESN-based time-series forecasting models have been proposed, including an improved topology ESN (IESN) [72] and a multireservoir ESN [73].

Transformer-based approaches have also demonstrated success in time-series analysis in industrial applications. For example, a gated convolutional transformer model was deployed for time-series forecasting in the chemical industry [22], whereas a model leveraging pretrained transformers was utilized for anomaly detection in semiconductor manufacturing [23]. Similarly, a TCN-based approach was used for fault detection in semiconductor circuit manufacturing, utilizing a multiple-channel CNN with each channel corresponding to a single sensor [74].

Feature extraction techniques play a pivotal role in industrial time-series applications by enabling the identification of critical patterns and relationships in high-dimensional, noisy data. Methods such as wavelet [75] and Fourier [76] transforms have been widely used for extracting frequency-domain features, while autoencoders [71] and attention

mechanisms [77] provide adaptive feature extraction in DL models.

Specifically within the antenna manufacturing industry, various process optimization frameworks have been introduced. An AI-based time-series anomaly detection framework was proposed to detect unusual fluctuations in sensor data reception rate [78]. Quality control approaches include a lightweight blockchain-enabled deep residual network method for defect detection [3], and a process optimization method combining finite element modeling and hybrid genetic-ant colony algorithms to improve the dimensional precision of riveted assemblies [79].

Despite significant advancements, the current state-of-the-art in time-series forecasting for industrial applications still faces several key limitations. Through our study, three main challenges were identified. As data complexity increases, many methods struggle to account for correlations between variables and across production stages, leading to suboptimal performance [80]. Moreover, the process of selecting and tuning the most suitable deep neural network remains a challenging and expertise-intensive task [81]. Furthermore, these methods frequently lack robust domain generalization, making them prone to performance degradation when faced with unseen, noisy, or heterogeneous data distributions typical in real-world industrial environments [82].

To address these challenges, this study introduces a simple, yet efficient, forecasting method. This method integrates information across multiple production stages and variables to enhance forecasting accuracy in complex industrial applications with temporal and spatial correlations. Additionally, the simplicity of the method reduces the need for laborious manual parameter tuning, speeding up the development process. Finally, unlike methods tested only on benchmark datasets, our approach is validated using two heterogeneous real-world use cases, highlighting its ability to deliver consistent performance across industrial domains.

III. Methodology

A. Problem Formulation

Time-series forecasting involves predicting future values based on previously observed data points. In this context, we define the problem for both single-variate and multivariate time-series forecasting.

1) Single-Variate Time-Series Forecasting: Given a sequence of historical observations $\{x_{t-n+1}, x_{t-n+2}, \ldots, x_t\}$, where $x_i \in \mathbb{R}$ represents the value at time step i, the goal of single-variate time-series forecasting is to forecast the future value x_{t+h} at a horizon h steps ahead. This can be formalized as finding a function f such that

$$\hat{x}_{t+h} = f(x_t, x_{t-1}, \ldots, x_{t-n+1}) \tag{1}$$

where $\hat{x}_{t+h}$ is the predicted value at time $t+h$, and n is the number of past observations used for prediction.

2) Multivariate Time-Series Forecasting: In the multivariate case, we consider multiple-related time series. Let $\mathbf{x}_t = [x_t^{(1)}, x_t^{(2)}, \ldots, x_t^{(m)}]^\top$ represent the vector of observations at time t across m variables. The goal is to forecast the future vector $\mathbf{x}_{t+h}$ based on the past n observations across all variables. The problem can be defined as

$$\hat{\mathbf{x}}_{t+h} = f(\mathbf{x}_t, \mathbf{x}_{t-1}, \ldots, \mathbf{x}_{t-n+1}) \tag{2}$$

where $\hat{\mathbf{x}}_{t+h} = [\hat{x}_{t+h}^{(1)}, \hat{x}_{t+h}^{(2)}, \ldots, \hat{x}_{t+h}^{(m)}]^\top$ is the predicted vector of values at time $t+h$.

3) Multivariate Time-Series Forecasting With Multistep Horizon: In multivariate time-series forecasting with a multistep horizon, the goal is to forecast a sequence of future vectors over a horizon H steps ahead. The prediction can be represented as a 2-D matrix, where rows correspond to different variables and columns correspond to different future time steps. Given the historical multivariate time series $\{\mathbf{x}_t, \mathbf{x}_{t-1}, \ldots, \mathbf{x}_{t-n+1}\}$, the objective is to forecast the matrix $\hat{\mathbf{X}}_{t+1:t+H}$

$$\hat{\mathbf{X}}_{t+1:t+H} = \begin{bmatrix} \hat{x}_{t+1}^{(1)} & \hat{x}_{t+2}^{(1)} & \cdots & \hat{x}_{t+H}^{(1)} \\ \hat{x}_{t+1}^{(2)} & \hat{x}_{t+2}^{(2)} & \cdots & \hat{x}_{t+H}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{x}_{t+1}^{(m)} & \hat{x}_{t+2}^{(m)} & \cdots & \hat{x}_{t+H}^{(m)} \end{bmatrix} \tag{3}$$

where $\hat{x}_{t+h}^{(i)}$ represents the predicted value for the ith variable at time step $t+h$, and the matrix $\hat{\mathbf{X}}_{t+1:t+H}$ has dimensions $m \times H$, with m being the number of variables and H being the number of future time steps to forecast.

This formulation allows the model to capture both temporal dependencies across time steps and correlations among multiple variables.

B. Proposed Architecture

Our proposed model architecture is a lightweight MLP-based framework designed to address both short-term and long-term forecasting challenges in time-series data. This architecture, as shown in Fig. 1, combines three specialized predictor blocks, each optimized for different temporal patterns. The first block employs the patching mechanism from PatchTST, which excels in capturing short-range dependencies by breaking down the time series into non-overlapping patches. The second and third blocks are dedicated to mid- and long-range feature extraction, utilizing MA and DFT seasonal decomposition techniques, respectively. In order to combine the outputs of these diverse components, an EWA module is integrated, which adaptively balances the contributions of each predictor block. This combination allows the model to deliver precise and robust forecasts across various time horizons, leveraging the strengths of each individual component to address the complexities of time-series data.

1) Patching Block: This block builds upon the work of Nie et al. [27] and their PatchTST model, which was originally developed for long-term time-series forecasting. In the original PatchTST architecture, a patching layer was introduced to segment the input sequence into shorter tokens, effectively reducing the token length and thereby decreasing the computational complexity associated with the Transformer-based backbone. Our proposed model leverages this patching mechanism to extract short-term features from the time series

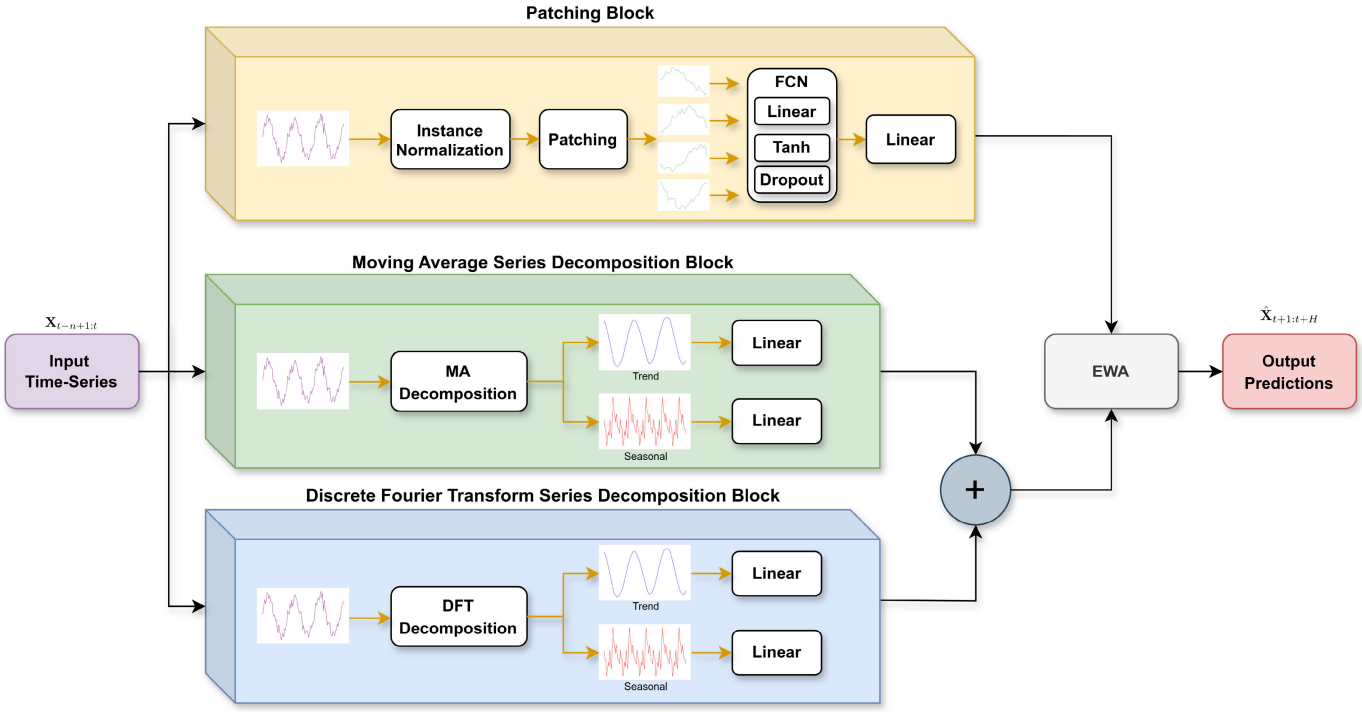


Fig. 1. Model architecture of PDFLinear$_{\text{EWA}}$ consisting of three predictor blocks. The patching block partitions the input time series into non-overlapping patches and processes them individually. The MA series decomposition block analyzes the input time series into its trend and seasonal components using an MA kernel, whereas the DFT series decomposition block works similarly, leveraging a DFT decomposition scheme. The EWA module fuses the outputs of the three predictor blocks using an exponentially weighting mechanism.

by focusing on individual patches. This approach allows the model to capture localized patterns efficiently, making it well-suited for applications where short-term dynamics play a crucial role.

The initial layer of this block employs the instance normalization technique introduced by Kim et al. [83] to address temporal distribution shifts commonly observed in time-series data. This mechanism operates on a sliding-window basis, where for each input window $\mathbf{X}_{t-n+1:t}$ and each variable i, the mean $\bar{x}_t^{(i)}$ and variance $\sigma_t^{(i)}$ are computed as follows:

$$\begin{aligned}\bar{x}_t^{(i)} &= \frac{1}{n}\sum_{j=0}^{n-1} x_{t-j}^{(i)} \\ \sigma_t^{(i)} &= \sqrt{\frac{1}{n}\sum\nolimits_{j=0}^{n-1}(x_{t-j}^{(i)} - \bar{x}_t^{(i)})^2}\end{aligned} \tag{4}$$

where $x_j^{(i)}$ refers to the value of the ith variable at time step j. Afterward, the input sliding window is normalized by subtracting its mean and dividing by its variance

$$z_t^i = \phi(x_t^i) = \frac{x_t^i - \bar{x}_t^{(i)}}{\sigma_t^{(i)}}. \tag{5}$$

To ensure that the model focuses on isolated sub-sequences of the input time series, the patching mechanism of PatchTST is applied to the normalized sequence, splitting it into sub-sequences of length l. Consequently, each variable i is divided into (n/l) non-overlapping sub-sequences

$$\mathbf{p}_{t-n+1:t}^{(i)} = \begin{bmatrix} \mathbf{p}_q^{(i)} = \{z_t^{(i)}, z_{t-1}^{(i)}, \ldots, z_{t-l+1}^{(i)}\} \\ \mathbf{p}_{q-1}^{(i)} = \{z_{t-l}^{(i)}, z_{t-l-1}^{(i)}, \ldots, z_{t-2l+1}^{(i)}\} \\ \vdots \\ \mathbf{p}_1^{(i)} = \{z_{t-(q-1)l}^{(i)}, \ldots, z_{t-ql+1}^{(i)}\} \end{bmatrix} \tag{6}$$

where $\mathbf{p}_{t-n+1:t}^{(i)}$ represents the patched input window for variable i, and $\mathbf{p}_q^{(i)}$ represents its qth patch, with $q \in \{1, 2, \ldots, (n/l)\}$.

After applying normalization and patching transformations, an MLP layer is employed to extract features from each sub-sequence and map each patch into a d-dimensional latent space representation. The activation function used is tanh, which confines the extracted features within the $[-1, 1]$ range, ensuring bounded and nonlinear outputs. To enhance the model's robustness and prevent overfitting, a dropout layer is incorporated, which regularizes the MLP's output by randomly deactivating a portion of the neurons during training

$$\mathbf{L}_q^{(i)} = \text{Dropout}\left(\text{Tanh}\left(\text{Linear}\left(\mathbf{p}_q^{(i)}\right)\right)\right) \tag{7}$$

where $\mathbf{L}_q^{(i)} \in \mathbb{R}^d$ is the extracted d-dimensional feature embedding of the qth patch and the ith variable. We use $d = 128$ as the default parameterization. It should be noted that the weights of the MLP are shared between all patches and variables, thus reducing model complexity. The embedded patches are then concatenated into a single vector, which is

then transformed via a linear module

$$\hat{\mathbf{Z}} = \text{Dropout}\left(\text{Linear}\left(\text{Concat}\left(\{L_q^{(i)}\}_{i=1,\ q=1}^{n,\ m}\right)\right)\right) \tag{8}$$

where $\hat{Z} \in R^{m\times H}$ is the non-normalized forecast of the patching block. Unlike the MLP layer described in (7), which focuses on individual variables, the linear layer in (8) considers correlations between variables and their interdependencies.

The final layer of the patching block applies instance denormalization (inverse of the instance normalization layer), producing the final forecast

$$\hat{x}_j^{(i)} = \frac{\hat{z}_j^{(i)} + \bar{x}_t^{(i)}}{\sigma_t^{(i)}} \tag{9}$$

$$\hat{\mathbf{X}}_{t+1:t+H} = \begin{bmatrix} \hat{x}_{t+1}^{(1)} & \hat{x}_{t+2}^{(1)} & \cdots & \hat{x}_{t+H}^{(1)} \\ \hat{x}_{t+1}^{(2)} & \hat{x}_{t+2}^{(2)} & \cdots & \hat{x}_{t+H}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{x}_{t+1}^{(m)} & \hat{x}_{t+2}^{(m)} & \cdots & \hat{x}_{t+H}^{(m)} \end{bmatrix} \tag{10}$$

where $\hat{x}_j^{(i)}$ is the final prediction of the patching block for the ith variable at the jth future time step and $\hat{\mathbf{X}}_{t+1:t+H}$ is the forecast matrix for all m variables and H future time steps.

2) Seasonal Decomposition Blocks: The seasonal decomposition blocks, in contrast to the patching block, focus on capturing mid-range and long-range dependencies present in the input time series. Their architecture utilizes two distinct time-series decomposition schemes: one based on MA decomposition and the other on DFT decomposition. The MA decomposition block follows the approach introduced by Wu et al. [53], smoothing each input time-series instance using an MA kernel. This smoothing process effectively isolates the underlying trend and seasonal components, reducing noise and enhancing the model's ability to capture long-term patterns. The decomposition is formulated as

$$\begin{aligned} \mathbf{X}_T^{\text{MA}} &= \text{AvgPool}\left(\text{Padding}\left(\mathbf{X}\right)\right) \\ \mathbf{X}_S^{\text{MA}} &= \mathbf{X} - \mathbf{X}_T^{\text{MA}} \end{aligned} \tag{11}$$

where $\mathbf{X}$ represents the input time-series instance, whereas Padding() ensures that the smoothed trend component retains the same length as the original time series. AvgPool() denotes the MA smoothing operation, with $\mathbf{X}_T^{\text{MA}}$ representing the smoothed trend component. The seasonal component $\mathbf{X}_S^{\text{MA}}$ is computed by subtracting the trend component from the original input time series. Both components are passed through separate linear modules to calculate their respective predictions

$$\begin{aligned} \hat{\mathbf{X}}_T^{\text{MA}} &= \text{Linear}_T^{\text{MA}}\left(\mathbf{X}_T^{\text{MA}}\right) \\ \hat{\mathbf{X}}_S^{\text{MA}} &= \text{Linear}_S^{\text{MA}}\left(\mathbf{X}_S^{\text{MA}}\right) \end{aligned} \tag{12}$$

where $\hat{\mathbf{X}}_T^{\text{MA}}$ and $\hat{\mathbf{X}}_S^{\text{MA}}$ are the trend and seasonal predictions, respectively. It is noted that each linear module processes a single variable at a time, with weights being shared between variables.

Similarly, the DFT decomposition block decomposes the input time-series instance into its trend and seasonal components by transforming it into the frequency domain and identifying the top k most dominant frequencies, i.e., the frequencies with the largest magnitudes. The original time series is reconstructed by removing the least dominant frequencies and applying the inverse DFT

$$\begin{aligned} \mathbf{X}_T^{\text{DFT}} &= \mathcal{F}^{-1}\left(\text{Select}_{\text{k}}\left(\mathcal{F}\left(\mathbf{X}\right)\right)\right) \\ \mathbf{X}_S^{\text{DFT}} &= \mathbf{X} - \mathbf{X}_T^{\text{DFT}} \end{aligned} \tag{13}$$

where $\hat{\mathbf{X}}_T^{\text{DFT}}$ represents the reconstructed trend component, $\mathcal{F}$ and $\mathcal{F}^{-1}$ represent the DFT and inverse DFT operations, respectively, and $\mathbf{X}_S^{\text{DFT}}$ represents the seasonal component calculated by subtracting the trend component from the original time series. As in the MA decomposition case, both components are passed through separate linear modules

$$\begin{aligned} \hat{\mathbf{X}}_T^{\text{DFT}} &= \text{Linear}_T^{\text{DFT}}\left(\mathbf{X}_T^{\text{DFT}}\right) \\ \hat{\mathbf{X}}_S^{\text{DFT}} &= \text{Linear}_S^{\text{DFT}}\left(\mathbf{X}_S^{\text{DFT}}\right). \end{aligned} \tag{14}$$

Finally, the outputs from both decomposition blocks are combined to produce the final prediction $\hat{\mathbf{X}}_{\text{Decomp}}$

$$\hat{\mathbf{X}}_{\text{Decomp}} = \hat{\mathbf{X}}_T^{\text{MA}} + \hat{\mathbf{X}}_S^{\text{MA}} + \hat{\mathbf{X}}_T^{\text{DFT}} + \hat{\mathbf{X}}_S^{\text{DFT}}. \tag{15}$$

By integrating the results from the MA and DFT decompositions, the model is able to capture both long-term trends and short-term seasonal patterns. This complementary approach enhances the model's ability to address both time-domain variations and frequency-based seasonalities, resulting in a more comprehensive and accurate understanding of complex temporal dynamics.

3) Exponentially Weighted Averaging: As previously discussed, the patching block focuses on capturing short-range dependencies, whereas the seasonal decomposition blocks are designed to capture mid-range and long-range. To effectively combine the predictions from these three blocks, we introduce a simple yet flexible weighting mechanism called EWA. EWA utilizes a learnable positive base factor β to generate exponentially decaying or increasing weights for each future time step

$$\mathbf{w} = \left[\beta^1, \beta^2, \ldots, \beta^H\right] \tag{16}$$

where the vector $\mathbf{w} \in \mathbb{R}^H$ contains the exponential weights for H future time steps. To ensure that the elements of $\mathbf{w}$ are constrained within the range [0, 1], we apply a normalization step

$$\mathbf{w}_{\text{norm}} = \frac{\mathbf{w} - \min\left(\mathbf{w}\right)}{\max\left(\mathbf{w}\right) - \min\left(\mathbf{w}\right)}. \tag{17}$$

Finally, the normalized weight vector $\mathbf{w}_{\text{norm}}$ is used to fuse the predictions of the patching block, denoted as $\hat{\mathbf{X}}_{\text{Patch}}$, and the seasonal decomposition blocks, denoted as $\hat{\mathbf{X}}_{\text{Decomp}}$. The final forecast $\hat{\mathbf{X}}$ is computed as

$$\begin{aligned} \hat{\mathbf{X}} &= \text{EWA}\left(\hat{\mathbf{X}}_{\text{Patch}}, \hat{\mathbf{X}}_{\text{Decomp}}\right) \\ &= \hat{\mathbf{X}}_{\text{Patch}} \odot \mathbf{w}_{\text{norm}} + \hat{\mathbf{X}}_{\text{Decomp}} \odot \left(\mathbf{w}_{\text{norm}} - 1\right). \end{aligned} \tag{18}$$

It is noted that for $\beta < 1$, the elements of the weight vector $\mathbf{w}$ decrease exponentially, assigning higher weights to the patching blocks's predictions for the earlier time steps

TABLE I
ASSEMBLY LINE STAGE DESCRIPTIONS

ID	Description
S1	Insertion of the left plastic housing (2D Pick & Place)
S2	Feeding of zamak chassis (2D Pick & Place), central supports of the antenna, plastic taps and insertion of them.
S3	Feeding of inferior lateral support of the antenna, bending and insertion, feeding of plastic taps and insertion
S4	Feeding of superior lateral support of the antenna, bending and insertion, feeding of plastic taps and insertion
S5	Insertion of the right plastic housing (2D Pick & Place)
S6	Feeding of plastic clips and zamak chassis. Placing of these elements. (2D Pick & Place)
S7	Feeding of dipoles and clamps, checking the dipole with artificial Vision and insertion of them
S8	Feeding of reflectors and insertion
S9	Another robot ties the antenna with an automatic tying system
S10	Packaging of the antenna in a bag and marking the bag with a videojet

and progressively lower weights for later time steps, favoring short-range predictions. Conversely, when $\beta > 1$, the weight vector increases exponentially, it assigns more importance to the seasonal decomposition blocks for short-range forecasts.

IV. EXPERIMENTS AND RESULTS

In this section, we present the evaluation dataset of the antenna assembly line and the applied preprocessing, the selected evaluation metrics, and the experimental setup. Additionally, the comparative performance results for all models are analyzed, and an ablation study for the proposed method is performed.

A. Datasets

To assess the effectiveness of our proposed forecasting method, we utilized two time-series datasets from the manufacturing sector. The primary dataset focuses on forecasting overall equipment effectiveness (OEE) metrics within a multi-stage antenna assembly line, aligning with the central objective of our study. To evaluate the generalizability of the proposed method in different manufacturing contexts, we additionally used a complementary dataset that contains time series data from a single-stage sink manufacturing process. Detailed descriptions of both datasets are provided in the following.

1) Antenna Assembly Line: The primary dataset used to evaluate and compare the forecasting models in this study comprises 345 days of OEE records from an antenna assembly line, including ten monitored stages. As presented in Table I and Fig. 2, for each of the ten stages, sensors record various operational and quality-related metrics, including time-based variables (active, waiting, error) and product quality indicators

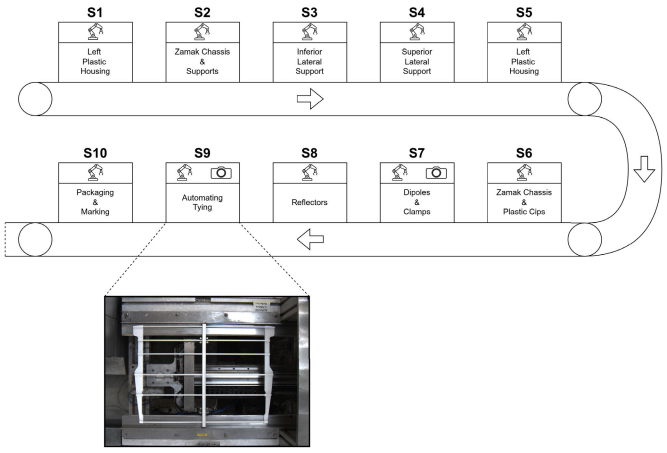


Fig. 2. Illustration of the antenna assembly line and its monitored stages. Stages with a camera icon are under visual inspection via computer vision algorithms, whereas OEE metrics are tracked across all stages.

(good, defective, missing items). These raw variables are subsequently used to calculate the OEE factors: availability (A), performance (P), and quality (Q), as defined below[1]

$$A = \frac{\text{Run Time}}{\text{Planned Production Time}} \tag{19}$$

$$P = \frac{\text{Ideal Cycle Time} \times \text{Total Count}}{\text{Run Time}} \tag{20}$$

$$Q = \frac{\text{Good Count}}{\text{Total Count}}. \tag{21}$$

Since the ideal cycle time is not available, we use the *Run Cycle*, defined as

$$\text{Run Cycle} = \frac{\text{Run Time}}{\text{Total Count}}. \tag{22}$$

This represents the average time it takes for an individual item to move through each stage of the process. Our focus is on the availability (A) and performance (P) factors, as the quality (Q) is generally constant ($Q = 1$) due to consistently low defect rates. Therefore, the quality factor does not provide any additional insights in this context.

To address the high sampling frequency (30 s per sample), the dataset is downsampled to 1-h intervals. Subsequently, it is split into training, validation, and test sets with an 80%, 10%, and 10% ratio, ensuring non-overlapping subsets. Each subset is then further divided into overlapping windows of length $n + H$, where $n = 168$ represents an input window of 168 h (or 7 days). The forecast horizon length, H, is evaluated across multiple values, with $H \in \{8, 24, 48, 72\}$.

2) Sink Manufacturing: The secondary dataset used for the evaluation of the proposed method originates from a sink manufacturing process and includes vibration sensor readings of manufacturing equipment from a single-stage process. The data span a two-month period and include five monitored variables.

1) v-rms: Root mean square of vibration velocity (mm/s).
2) a-Peak: Peak acceleration $\left(\text{m/s}^2\right)$.

[1] https://www.oee.com/calculating-oee/

3) a-rms: Root mean square of acceleration $\left(\mathrm{m/s^2}\right)$.
4) Crest: Crest factor of vibrations (1).
5) Temp: Temperature (°C).

Several preprocessing steps were applied prior to model training. Initially, the v-rms variable was excluded from the analysis due to low variability, following a near-bimodal distribution, as it did not provide meaningful information for the intended purpose of forecasting. Additionally, periods of inactivity, during which the manufacturing process was not operational, were removed from the dataset. Due to the high sampling rate of the original dataset, one reading every 5 s, the time series were downsampled to 1-h intervals.

Finally, the dataset was split into non-overlapping training, validation, and test sets with a 70%, 15%, and 15% ratio. The higher test set ratio, compared to the antenna assembly line dataset, was selected to account for the smaller dataset size, ensuring more reliable evaluation metrics. Similarly with the antenna assembly line dataset, it is divided into overlapping windows of length $n + H$, with $n = 168$ and $H \in \{8, 24, 48, 72\}$.

B. Evaluation Metrics

The evaluation metrics that were used to access the predictive performance of the selected methods are the root-mean-square error (RMSE), the symmetric mean absolute percentage error (SMAPE), the R^2, and the mean directional accuracy (MDA). MDA is not widespread in benchmark tests, but it is a useful metric in noisy datasets, where qualitative predictive analysis (e.g., capturing the trend) is important for production scheduling [84]. At time step t, for a forecast horizon of length H, and a forecast window of $\hat{\mathbf{X}}_{t+1:t+H}$, these metrics are calculated as follows:

$$\mathrm{RMSE} = \sqrt{\frac{1}{H}\sum_{i=t}^{H}(x_i - \hat{x}_i)^2} \tag{23}$$

$$\mathrm{SMAPE} = \frac{1}{H}\sum_{i=t}^{H}\frac{|x_i - \hat{x}_i|}{\frac{|x_i|+|\hat{x}_i|}{2}} \tag{24}$$

$$\mathrm{R}^2 = 1 - \frac{\sum_{i=t}^{H}(x_i - \hat{x}_i)^2}{\sum_{i=t}^{H}(x_i - \bar{x})^2} \tag{25}$$

$$\mathrm{MDA} = \begin{cases} 1, & \text{if } (x_{t+H} - x_t)(\hat{x}_{t+H} - \hat{x}_t) > 0 \\ 0, & \text{else} \end{cases} \tag{26}$$

In the case of MDA, since we are dealing with multistep forecast horizons, the sign for each window is computed by subtracting the first value of the forecast window from the last value. The final metric value is calculated by averaging across all forecast windows.

C. Experimental Setup

The implementation of our model was done in Python v3.10, PyTorch v2.3.0, and CUDA v.12.4. The model training and evaluation were conducted on a dedicated server with an Intel i9-13900K at 3 GHz, 64 GB DDR4 RAM, and two NVIDIA GeForce RTX 3080 Ti. The ADAM optimizer was selected for training with a learning rate of 1×10^{-4}. The batch size was 32, with a maximum of 30 epochs and early stopping patience of 3.

The implementations of most of the evaluated models were provided by the repositories of TimesNet [85] and time-series library.[2] The implementation of the patching block was based on PatchTST[3] [27], the implementation of the MA seasonal decomposition block was based on Autoformer[4] [53], and the implementation of the DFT seasonal decomposition block was based on TimeMixer[5] [17].

When available, the default recommended parameter configurations were used for each model architecture. Otherwise, as in the case of RNNs, various configurations were tested and evaluated via grid search and the use of the validation set, with the most performant ones being reported in the final comparison.

D. Comparative Results

In this section, we evaluate the baseline models on the benchmark dataset and compare them against our proposed method in terms of predictive performance and required computational resources. This section is further split into three subsections, including the quantitative results, the qualitative results, and the computational requirements results.

Representative state-of-the-art models of each architecture class were selected as baseline models.

1) RNNs: LSTM [35], GRU [36], bi-LSTM, biGRU [37], FiLM [43], and Mamba [48].
2) MLPs: Linear, NLinear and DLinear [16], LightTS [33], TimeMixer [17], TiDE [86], FreTS [31], Koopa [32], and FreTS [31].
3) Transformers: canonical Transformer [49], Informer [52], Reformer [51], Autoformer [53], FEDformer [28], PatchTST [27], and iTransformer [25].
4) TCNs: MICN [57] and TimesNet [58].

Additionally, we include the Naive model, as typically followed in comparison studies [16], which outputs the last input value as a prediction for all future time steps, as the minimum baseline for a model to be considered useful.

1) Quantitative Results: In Table II, the comparative evaluation results of each model for the antenna asembly line dataset are reported, including the RMSE, SMAPE, R^2, and MDA for four different forecast horizons (8 H, 24 H, 48 H, 72 H). Lower RMSE and SMAPE indicate higher forecasting performance, whereas lower R^2 and MDA indicate the opposite. In general, there does not seem to exist a noticeable difference between the four architecture classes. This indicates that the choice of the underlying architecture (MLP, recurrent, TCN, Transformer) does not contribute significantly to the performance of the models. Instead, the performance variations are mainly derived from the early feature extraction layers. Indeed, the models with no feature extraction layers (LSTM/GRU, Linear, Transformer) perform the worst in each architecture class. As indicated by previous studies, the early feature extraction layers serve two key purposes: 1) they reduce noise in the input time series, helping to prevent overfitting during training; and

[2]https://github.com/thuml/Time-Series-Library
[3]https://github.com/PatchTST/PatchTST
[4]https://github.com/thuml/Autoformer
[5]https://github.com/kwuking/TimeMixer

TABLE II
COMPARISON OF FORECASTING MODELS-ANTENNA ASSEMBLY LINE DATASET

Model	8 Hours				24 Hours				48 Hours				72 Hours			
	RMSE	SMAPE	R2	DA	RMSE	SMAPE	R2	DA	RMSE	SMAPE	R2	DA	RMSE	SMAPE	R2	DA
Naive	0.210	0.348	0.586	0.075	0.227	0.414	0.518	0.040	0.228	0.422	0.514	0.024	0.230	0.427	0.506	0.061
LSTM	0.190	0.379	0.662	0.470	0.190	0.379	0.663	0.434	0.188	0.372	0.670	0.512	0.185	0.359	0.679	0.467
bi-LSTM	0.192	0.382	0.654	0.458	0.191	0.382	0.658	0.497	0.190	0.376	0.664	0.450	0.187	0.362	0.672	0.477
GRU	0.190	0.372	0.662	0.448	0.190	0.377	0.661	0.435	0.188	0.370	0.669	0.487	0.184	0.351	0.683	0.455
biGRU	0.181	0.361	0.693	0.520	0.194	0.391	0.649	0.458	0.199	0.388	0.632	0.449	0.191	0.354	0.659	0.476
FiLM (2022) [43]	0.171	0.328	0.725	0.570	0.176	0.330	0.710	0.652	0.186	0.329	0.676	0.627	0.199	0.409	0.630	0.515
Mamba (2023) [48]	0.175	0.337	0.712	0.482	0.177	0.345	0.708	0.465	0.175	0.340	0.713	0.464	0.174	0.328	0.717	0.455
Linear (2022) [16]	0.218	0.427	0.556	0.431	0.220	0.436	0.546	0.494	0.214	0.417	0.572	0.452	0.209	0.399	0.590	0.502
NLinear (2022) [16]	0.230	0.455	0.506	0.434	0.241	0.488	0.455	0.482	0.244	0.473	0.446	0.435	0.245	0.471	0.440	0.512
DLinear (2022) [16]	0.180	0.351	0.696	0.432	0.183	0.359	0.687	0.579	0.181	0.350	0.695	0.542	0.178	0.336	0.702	0.471
LightTS (2022) [33]	0.215	0.421	0.567	0.488	0.212	0.418	0.578	0.502	0.211	0.412	0.585	0.493	0.211	0.403	0.585	0.489
$\text{TimeMixer}_{\text{MA}}$ (2023) [17]	0.177	0.339	0.708	0.533	0.177	0.348	0.707	0.597	0.176	0.341	0.710	0.622	0.179	0.331	0.700	0.563
$\text{TimeMixer}_{\text{DFT}}$ (2023) [17]	0.177	0.341	0.707	0.537	0.176	0.346	0.709	0.584	0.176	0.341	0.710	0.628	0.179	0.331	0.700	0.562
TiDE (2024) [86]	0.196	0.382	0.639	0.415	0.192	0.378	0.654	0.465	0.192	0.376	0.655	0.516	0.193	0.367	0.652	0.509
Koopa (2023) [32]	0.178	0.346	0.703	0.496	0.179	0.350	0.700	0.487	0.177	0.342	0.709	0.526	0.175	0.331	0.713	0.457
FreTS (2023) [31]	0.199	0.382	0.629	0.555	0.216	0.410	0.563	0.636	0.212	0.399	0.579	0.630	0.204	0.376	0.610	0.570
Transformer (2017) [49]	0.210	0.426	0.587	0.485	0.212	0.432	0.581	0.501	0.198	0.389	0.633	0.494	0.190	0.364	0.663	0.457
Informer (2021) [52]	0.221	0.413	0.545	0.491	0.200	0.385	0.627	0.467	0.194	0.373	0.649	0.465	0.193	0.363	0.653	0.441
Reformer (2020) [51]	0.194	0.391	0.648	0.501	0.193	0.386	0.650	0.495	0.191	0.376	0.659	0.483	0.189	0.362	0.664	0.481
Autoformer (2022) [53]	0.187	0.367	0.672	0.484	0.185	0.364	0.680	0.478	0.183	0.351	0.689	0.499	0.179	0.335	0.699	0.467
FEDformer (2022) [28]	0.186	0.365	0.675	0.463	0.183	0.357	0.686	0.466	0.180	0.348	0.699	0.483	0.180	0.341	0.698	0.469
PatchTST (2023) [27]	0.171	0.331	0.727	0.555	0.176	0.345	0.710	0.624	0.176	0.340	0.713	0.600	0.174	0.329	0.715	0.535
iTransformer (2023) [25]	0.179	0.348	0.700	0.499	0.184	0.362	0.683	0.535	0.184	0.358	0.686	0.510	0.184	0.349	0.683	0.456
MICN (2023) [57]	1.337	0.754	-15.738	0.471	0.352	0.428	-0.162	0.536	0.182	0.339	0.692	0.541	0.169	0.317	0.732	0.479
TimesNet (2023) [58]	0.187	0.365	0.673	0.496	0.188	0.360	0.671	0.478	0.181	0.348	0.696	0.515	0.178	0.335	0.703	0.405
$\text{PDFLinear}_{\text{EWA}}$	0.169	0.329	0.733	0.570	0.173	0.339	0.718	0.627	0.173	0.334	0.722	0.615	0.170	0.322	0.728	0.578

2) they extract valuable features that might be challenging for the models to capture otherwise. These factors suggest that very complex DL methods with a high parameter count suffer from high variance, leading to underperformance, particularly in cases of low-dimensional and/or noisy data.

Within the recurrent class, it is observed that FiLM outperforms all other models for short-term forecast horizons (8 H, 24 H), confirming the efficacy of its FT block. Mamba, on the other hand, excels at long-term forecast horizons (48 H, 72 H), while maintaining good performance in short-term predictions. In the MLP class, TimeMixer and Koopa outperform all other models across all forecast horizons, underlying the importance of their respective time-series decomposition schemes. In the case of DLinear, the inclusion of the MA decomposition mechanism, without any other parameter addition, greatly enhances performance over the simple linear model, illustrating the significance of effective feature extraction mechanisms.

The top performance in the Transformer class is achieved by PatchTST across all forecast horizons due to its patching layer, which is also utilized in our proposed method. The canonical Transformer appears to overfit the training data, compared to PatchTST and iTransformer, whose dimensionality reduction schemes allow them not only to reduce model complexity but also to provide a regularizing effect. This indicates that attention-based models overfit considerably in the absence of noise reduction or regularization mechanisms. FEDformer and Autoformer, leveraging their respective DFT and MA decomposition schemes, exhibit comparable results, falling below the two models mentioned above. It is important to highlight that multiple encoder–decoder configurations were evaluated for each model, and in all cases, the configurations with fewer encoder–decoder blocks consistently achieved the highest performance. Consequently, the results presented in Table II correspond to configurations with a single encoder and a single decoder block, if applicable.

In the case of TCNs, TimesNet exhibits mediocre results comparable to the median Transformer or MLP. Interestingly, the results shown in Table II correspond to the simpler

TABLE III
COMPARISON OF FORECASTING MODELS-SINK MANUFACTURING DATASET

Model	8 Hours				24 Hours				48 Hours				72 Hours			
	RMSE	SMAPE	R2	DA	RMSE	SMAPE	R2	DA	RMSE	SMAPE	R2	DA	RMSE	SMAPE	R2	DA
Naive	2.880	0.309	0.960	0.000	2.952	0.307	0.958	0.000	2.970	0.311	0.957	0.000	2.993	0.318	0.957	0.000
FiLM (2022) [43]	2.239	0.244	0.976	0.642	2.241	0.240	0.976	0.519	2.414	0.245	0.972	0.601	3.123	0.343	0.953	0.432
$\text{TimeMixer}_{\text{MA}}$ (2023) [17]	2.313	0.258	0.975	0.583	2.364	0.256	0.973	0.521	2.425	0.257	0.972	0.613	2.422	0.280	0.972	0.529
PatchTST (2023) [27]	2.328	0.255	0.974	0.604	2.373	0.261	0.973	0.537	2.484	0.271	0.970	0.517	2.479	0.275	0.970	0.529
TimesNet (2023) [58]	2.426	0.264	0.972	0.575	2.381	0.267	0.973	0.528	2.445	0.273	0.971	0.479	2.426	0.280	0.971	0.542
$\text{PDFLinear}_{\text{EWA}}$	2.303	0.251	0.975	0.604	2.356	0.259	0.973	0.565	2.381	0.263	0.973	0.597	2.360	0.268	0.973	0.655

configuration of TimesNet, with just a single TimesBlock, which achieved the highest performance. This again confirms that higher model complexity results in overfitting and reduced forecasting performance. The most interesting results are displayed by MICN, which in short-term forecast horizons completely fails, whereas in long-term forecast horizons exhibits the best results of all forecasting models, with $\text{PDFLinear}_{\text{EWA}}$ a close second. This indicates that in the case of TCNs, careful manual parameter configuration and adjustment for input length, forecast horizon, and domain-specific characteristics are paramount in optimizing performance.

Finally, $\text{PDFLinear}_{\text{EWA}}$ achieves the best results in short- and mid-term forecast horizons (8 H, 24 H, 48 H), while also achieving comparable results to the best models in long-term forecast horizons (72 H). These results confirm the effectiveness of our proposed model in its ability to adaptively adjust to different forecast scenarios, optimizing performance across varying time horizons. Additionally, these observations reinforce the hypothesis that specialized time-series feature extraction and decomposition techniques are crucial for enhancing model robustness and accuracy, especially in complex, nonlinear time series. Overall, the results underscore the importance of aligning the choice of model architecture and feature extraction strategy with the specific characteristics of the forecasting task at hand, as this alignment significantly impacts the overall effectiveness and reliability of the predictions.

In Table III, the performance of six forecasting models is evaluated for the sink manufacturing dataset across four forecast horizons (8 H, 24 H, 48 H, 72 H), using the same metrics (RMSE, SMAPE, R^2, and DA). The results highlight the competitive performance of $\text{PDFLinear}_{\text{EWA}}$, which consistently approaches or outperforms the other models, particularly at longer forecast horizons (72 H). FiLM demonstrates better results at shorter horizons, leveraging its FT block, but exhibits reduced effectiveness at longer horizons. TimeMixer and PatchTST deliver balanced performance, while TimesNet shows reliable, albeit average, results. These findings align with those observed in the antenna assembly line dataset, demonstrating the generalization capability of the proposed method across heterogeneous sectors and forecasting applications.

2) Qualitative Results: In Figs. 3 and 4, some indicative prediction examples are shown, for forecast horizons $H \in \{8, 24\}$ and $H \in \{48, 72\}$, respectively. To reduce noise typical of industrial time series, we compute and plot the ground-truth trend, offering a clearer, more informative representation of the data. Although our model may not align perfectly with the ground truth at every time step, its ability to closely follow the overall trend helps maintain a low RMSE compared to the other models.

Accurately forecasting the trend component enables more informed decision-making compared to relying solely on point-by-point forecast comparisons. For example, in Fig. 3, for forecast horizon $H = 24$, the ground-truth trend of the availability variable starts at approximately $A \approx 0.4$ and increases by $\Delta A_{gt} \approx +0.25$ over the next day. Even though the forecast values for this variable are lower than the ground truth, at both the start and the end, the positive trend is accurately captured with a forecast $\Delta A_f \approx +0.33$. In this case, the upward trend in A indicates that no immediate predictive maintenance is necessary, as the variable is expected to naturally reach acceptable levels, reducing error or waiting times. Conversely, the accurately forecast positive trend in the Run Cycle suggests an increase in the time required to produce each unit, highlighting the need for potential equipment recalibration at this stage. For longer forecast horizons, as shown in Fig. 4, a gradual performance degradation is observed, which is expected given the inherent challenges of forecasting overextended timeframes. Notably, abrupt drops in the Run Cycle variable are not fully captured, reflecting the difficulty in modeling sudden, nonlinear changes overextended periods.

It should be noted that, in order to maintain clarity, models such as PatchTST and TimeMixer, which achieve results very similar to our proposed model, are not included in the prediction plots. In any case, our model stands out from the other state-of-the-art models due to its simplicity and lower computational complexity, while still achieving the same or better forecasting performance.

3) Computational Requirements: In Table IV, we present the computational resource requirements for the top-performing models in each category. The metrics include the average inference time per sample, the parameter count

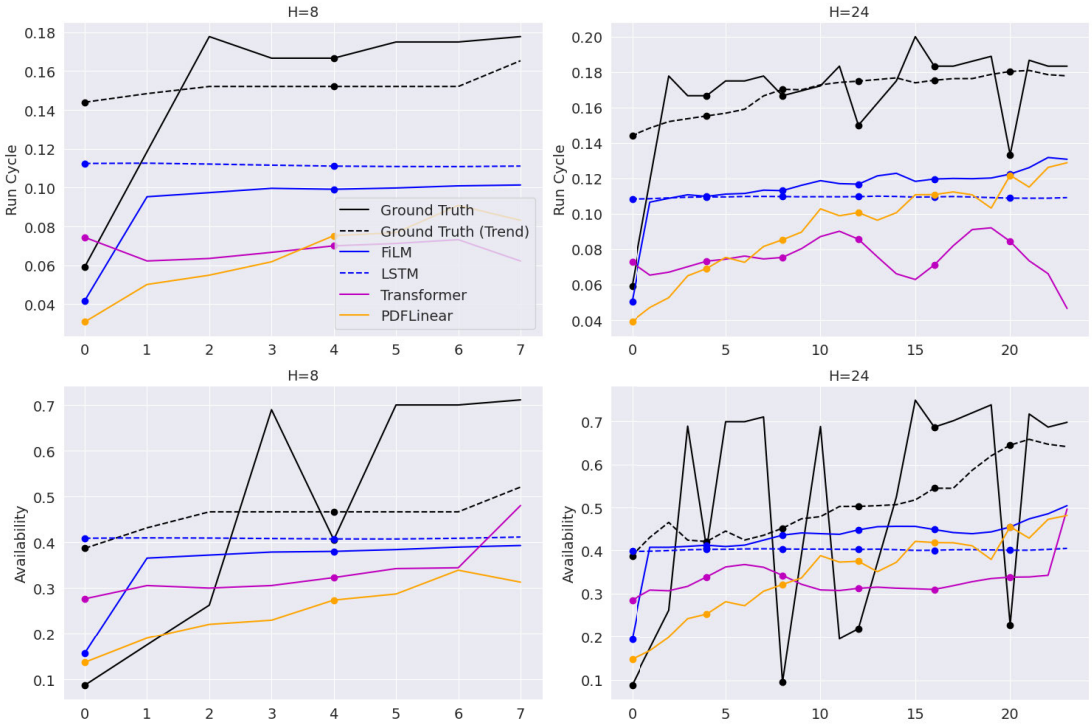


Fig. 3. Examples of indicative short-term forecasts from the best models of each architecture class for Stage S9 of the antenna assembly line dataset. The first row shows results for the Run Cycle variable, and the second row for the availability variable. The columns represent different forecast horizons: the first column corresponds to $H = 8$, whereas the second column corresponds to $H = 72$. The dashed black line represents the MA trend of the ground truth, reducing noise in the original time series for clearer and more informative comparison.

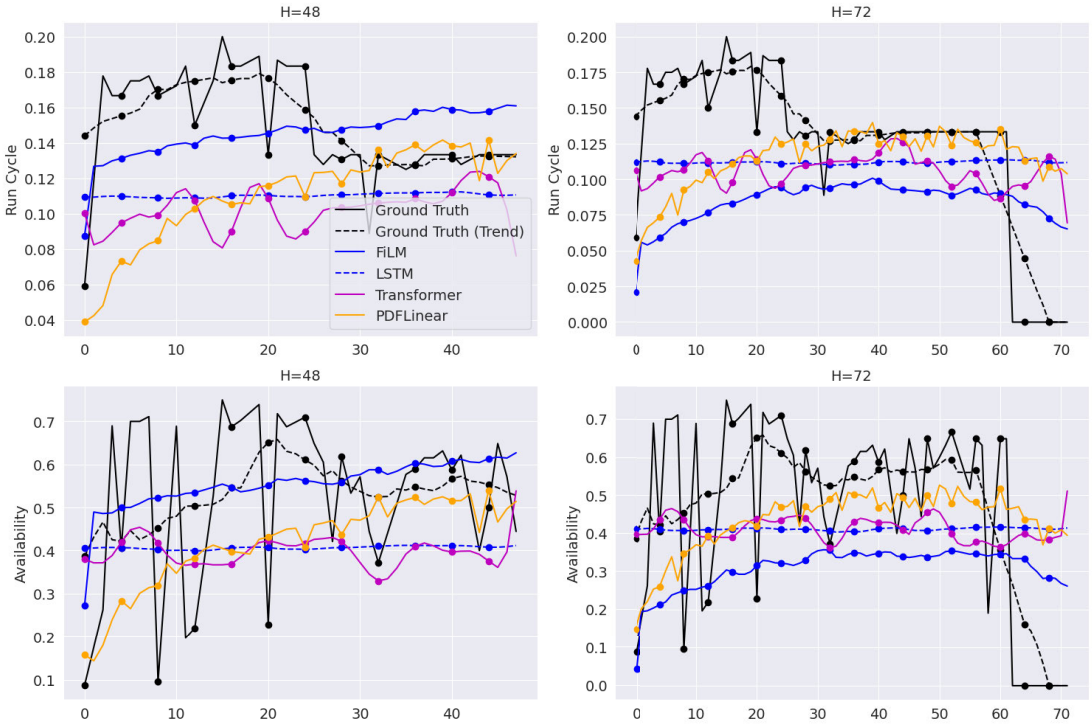


Fig. 4. Examples of indicative long-term forecasts from the best models of each architecture class for Stage S9 of the antenna assembly line dataset. The first row shows results for the Run Cycle variable, and the second row for the availability variable. The columns represent different forecast horizons: the first column corresponds to $H = 48$, whereas the second column corresponds to $H = 72$. The dashed black line represents the MA trend of the ground truth, reducing noise in the original time series for clearer and more informative comparison.

of each model, the FLOPs count per batch, and the GPU memory utilization in MBs, all measured during the testing phase for a forecast horizon of $H = 72$, representing a worst case scenario. Our proposed model outperforms most models across all metrics, except for PatchTST which has slightly lower inference times, but higher parameter

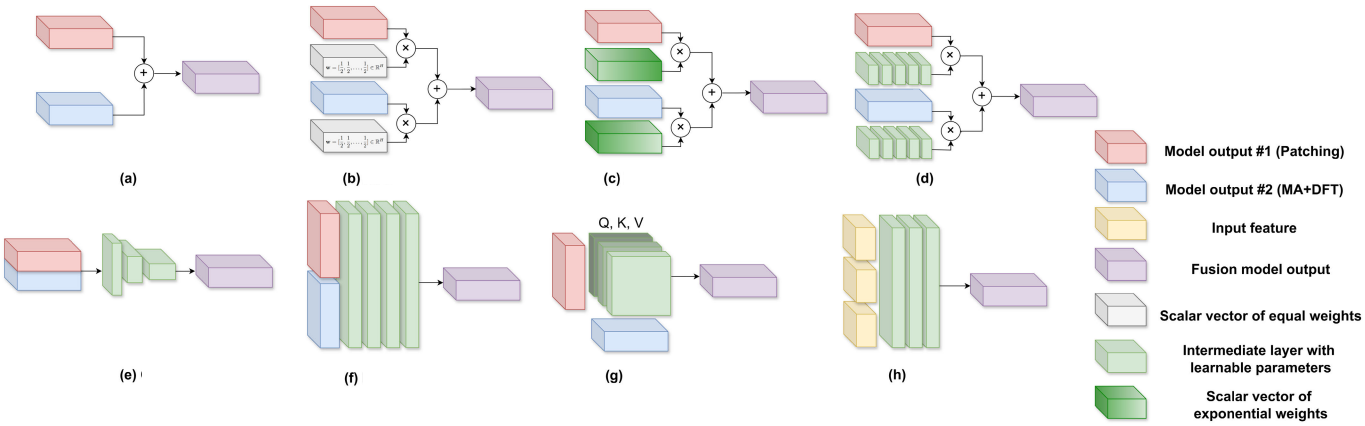


Fig. 5. Illustration of late and early fusion techniques evaluated in the ablation study. (a) ADD. (b) AVG. (c) EWA. (d) WA. (e) CNN. (f) MLP. (g) ATN. (h) Early fusion.

TABLE IV
COMPARISON OF FORECASTING MODELS' COMPUTATIONAL REQUIREMENTS

	PDFLinear	TimeMixer	FiLM	PatchTST	MICN
Time (ms)	0.883	2.81	12.7	0.841	1.60
#Parameters	$244K$	$245K$	$12,582K$	$295K$	$1,712K$
#FLOPs	$5.7M$	$38.7M$	$1009.7M$	$49.1M$	$70.8M$
GPU Mem.	51.4MB	117.5MB	1348.0MB	185.2MB	271.1MB

count, and significantly higher FLOPs count and memory utilization.

E. Ablation Study

To evaluate the contribution of each component of our proposed forecasting model, we conducted an ablation study. This analysis systematically removes or alters key elements of the model to measure their impact on performance. Specifically, we examined the effect of removing the patching block, the MA decomposition block, or the DFT decomposition block, either individually or in pairs. We did not include the case where only the MA decomposition block is retained, as this configuration corresponds to the DLinear model [16], whose results are shown in Table II. Additionally, we examined the effect of replacing the EWA module [Fig. 5(c)] with various fusion techniques, ranging from simple addition to attention-based fusion, as well as a simple early fusion technique. In the following list, we present the evaluated fusion techniques, ordered from simplest to most complex. The constituent models refer to: 1) the patching block and 2) the summed output of the MA and DFT blocks.

1) *Addition (ADD)*: This is the simplest fusion technique, where the outputs of each constituent model are summed for each time step. This approach does not introduce additional parameters or complexity, treating each model's output equally [Fig. 5(a)].
2) *Average (AVG)* [87]: A normalized version of ADD, this technique averages the outputs of the constituent models at each time. This approach ensures equal contribution from each model [Fig. 5(b)].
3) *Weighted average (WA)*: This fusion approach uses a learnable weight vector $\mathbf{w} \in (0, 1)^H$, restricted to the interval $(0, 1)$, to adaptively weight each constituent model's output at each time step. Although similar to the proposed EWA method, the dimensionality of the weight vector allows the outputs of each constituent model to be mixed with a unique weight at each time step (H), offering greater flexibility than EWA [Fig. 5(d)].
4) *CNN* [88]: This approach uses a shallow 1-D CNN to weight the outputs of each constituent model, in a sliding window approach, enabling information integration across adjacent time steps. Variables are treated as kernel channels, making the convolutional kernel's shape $2 \times k$, where 2 refers to the two constituent models and k the kernel size. For our case, we use $k = 5$ with a convolution stride $s = 1$. To preserve the output dimensions, zero-padding of $P = (k - 1/2) = 2$ is applied [Fig. 5(e)].
5) *MLP* [29]: This fusion approach leverages a shallow MLP to learn complex, nonlinear combinations of the constituent model outputs at each time step. The MLP takes as input the concatenated outputs of all constituent models for each variable and time step. In our experiments, we evaluated one and two hidden layers configurations, as the increase of layer count resulted in decreased performance [Fig. 5(f)].
6) *Attention-based (ATN)* [89]: Inspired by Transformer models, this technique dynamically adjusts the influence of each constituent model using an attention mechanism. Each attention head employs three weight matrices—query (Q), key (K), and value (V)—to compute relevance scores and adaptively combine the model outputs. To mitigate overfitting, the approach utilizes a single attention layer with two heads [Fig. 5(g)].
7) *Early fusion:* In our early fusion approach, a three-layer MLP is used to integrate intermediate features generated by each constituent model (e.g., patches from the patching block, and the trend and seasonal components from the decomposition blocks). These features are concatenated across all predictor blocks for each variable and time step before being fed into the MLP [Fig. 5(h)].

TABLE V
ABLATION STUDY

Components	Fusion	8 Hours				24 Hours				48 Hours				72 Hours			
		RMSE	SMAPE	R2	DA	RMSE	SMAPE	R2	DA	RMSE	SMAPE	R2	DA	RMSE	SMAPE	R2	DA
Patch	-	0.169	0.331	0.732	0.564	0.175	0.345	0.713	0.652	0.175	0.340	0.714	0.625	0.174	0.329	0.717	0.564
DFT	-	0.176	0.339	0.712	0.509	0.176	0.342	0.712	0.450	0.176	0.339	0.713	0.463	0.173	0.328	0.720	0.493
MA+DFT	ADD	0.174	0.336	0.717	0.527	0.174	0.340	0.718	0.570	0.173	0.334	0.721	0.539	0.170	0.321	0.729	0.528
Patch+DFT	ADD	0.177	0.339	0.708	0.554	0.177	0.344	0.707	0.627	0.173	0.332	0.722	0.617	0.168	0.316	0.736	0.567
Patch+MA	ADD	0.170	0.333	0.730	0.560	0.175	0.343	0.714	0.635	0.175	0.338	0.716	0.623	0.173	0.327	0.721	0.568
Patch+MA+DFT	ADD	0.175	0.335	0.713	0.576	0.176	0.344	0.709	0.643	0.173	0.332	0.722	0.607	0.168	0.317	0.735	0.578
Patch+MA+DFT	AVG	0.171	0.334	0.725	0.544	0.174	0.342	0.716	0.642	0.174	0.338	0.716	0.598	0.172	0.325	0.723	0.559
Patch+MA+DFT	WA	0.171	0.333	0.727	0.552	0.175	0.343	0.715	0.651	0.175	0.339	0.715	0.612	0.173	0.326	0.720	0.566
Patch+MA+DFT	CNN	0.256	0.522	0.387	0.480	0.209	0.401	0.591	0.628	0.237	0.479	0.474	0.632	0.174	0.325	0.716	0.528
Patch+MA+DFT	MLP(1)	0.203	0.404	0.613	0.516	0.204	0.406	0.612	0.510	0.220	0.435	0.549	0.479	0.199	0.369	0.631	0.434
Patch+MA+DFT	MLP(2)	0.204	0.402	0.609	0.500	0.206	0.411	0.602	0.466	0.215	0.412	0.570	0.453	0.202	0.374	0.618	0.441
Patch+MA+DFT	ATN	0.194	0.376	0.646	0.485	0.210	0.394	0.589	0.483	0.192	0.372	0.657	0.540	0.194	0.361	0.647	0.466
Patch+MA+DFT	Early	0.194	0.376	0.648	0.507	0.195	0.383	0.643	0.525	0.193	0.373	0.654	0.516	0.189	0.358	0.665	0.471
Patch+MA+DFT	EWA	0.169	0.329	0.733	0.570	0.173	0.339	0.718	0.627	0.173	0.334	0.722	0.615	0.170	0.322	0.728	0.578

The ablation study results, as presented in Table V, highlight the performance of various model configurations. For the 8-h forecast horizon, the base patch model delivers the best results among the other base models, validating the hypothesis that the patching mechanism effectively captures short-range dependencies. Among the fusion techniques, ADD, AVG, and WA exhibit consistent performance across all forecast horizons. However, they underperform relative to the proposed EWA mechanism, particularly for short-term forecasts. Interestingly, the more complex fusion techniques, such as CNN, MLP, and ATN, underperform across all forecast horizons compared to the simpler methods. This underperformance can likely be attributed to overfitting, stemming from their higher parameter count and limited ability to generalize to unseen data.

Our proposed method (Patch+MA+DFT) leveraging the EWA module achieves the best overall performance. Across the 8-, 24-, and 48-h forecast horizons, it consistently outperforms other configurations, demonstrating its robustness in short- and mid-term forecasting scenarios. However, for the 72-h forecast horizon, models leveraging the DFT decomposition scheme slightly outperform our approach. This suggests that while DFT excels at capturing long-term dependencies, the EWA module may overemphasize short-term fluctuations, resulting in a marginal decline in accuracy over longer horizons. Nevertheless, the superior performance in shorter-term forecasting demonstrates the overall effectiveness and consistency of the EWA module.

V. DISCUSSION

The findings of this study highlight the efficacy of the proposed method in addressing the challenges of predictive maintenance and process optimization in manufacturing, which are pivotal procedures for ZDM. Concretely, we used historical data from OEE to forecast future production efficiency values, both short- and long-term, which provides valuable insights into the production, enabling more efficient equipment maintenance scheduling.

Experiments on the antenna assembly line dataset demonstrated our proposed method's robust performance across various forecast horizons, with specialized decomposition techniques (MA, DFT) and the EWA fusion mechanism being critical in enhancing forecasting accuracy. Our model achieved the highest performance for forecast horizons $H \in \{8, 24, 48\}$, with RMSE values of $\{0.169, 0.173, 0.173\}$, respectively. Although MICN slightly outperformed our model for $H = 72$, with an RMSE reduction of 0.01, our method demonstrated top overall performance across the other forecast horizons, where MICN struggled, or completely failed in the cases of $H \in \{8, 24\}$.

In the sink manufacturing dataset, the results closely resembled those observed in the antenna assembly line dataset. While our method ranked second for $H \in \{8, 24\}$, it achieved the best performance for $H \in \{48, 72\}$, highlighting its adaptability across varying forecast horizons. Even though it underperformed compared to FiLM in the short-term forecast horizons, the substantial performance gap for $H = 72$, with an RMSE of 2.360 for our method compared to 3.123 for FiLM, further demonstrates the robustness of our approach in handling longer forecast horizons effectively.

Generally, it was observed that simpler models, which leverage dimensionality reduction or regularization techniques, tend to outperform more complex models with a larger number of parameters, especially when training data are limited or noisy. This was particularly evident in the Transformer class of models, where architectures that incorporated dimensionality reduction and also reduced the encoder–decoder layers (e.g., PatchTST, iTransformer) showed greater performance stability. In contrast, larger, over-parameterized Transformer models often struggled with convergence and required more

computational resources, without necessarily yielding significantly better results.

Temporal feature extraction mechanisms improved performance across all architecture classes, as in the case of FiLM in the recurrent class, DLinear and TimeMixer in the MLP class, and Autoformer and FEDformer in the Transformer class. These results align with existing literature, reinforcing the importance of time-series feature extraction and denoising, as well as the potential of adaptive weighting mechanisms in fusing multiple forecasting models to boost performance in complex industrial environments.

However, some limitations were observed, particularly in long-term forecasting scenarios, where in some cases, simpler fusion techniques outperformed the proposed method. This suggests a need for refining the EWA mechanism to better balance short- and long-term dependencies. Additionally, while the model excelled in structured and static environments, its adaptability to noisy or dynamic industrial contexts remains to be explored. Finally, the second dataset, which represented a single-stage manufacturing process, aligns with datasets commonly used in predictive maintenance research, providing a valuable, yet limited, scope for assessing generalizability.

Future research should aim to extend the proposed method to a broader range of applications, both within and beyond the industrial manufacturing context. This includes exploring its adaptability to other domains, such as energy systems, healthcare, and supply chain management, where the integration of exogenous variables such as environmental factors, demand fluctuations, and resource availability plays a significant role in predictive accuracy [90].

VI. Conclusion

In this study, we explored the application of advanced DL architectures for time-series forecasting of OEE in a real-world antenna assembly line. By leveraging the strengths of various model architectures, including MLPs, recurrent models, Transformer-based models, and TCNs, we aimed to address the complexity of forecasting OEE metrics in multistage production environments.

Our proposed method, a lightweight, late-fusion MLP framework with an EWA fusion mechanism, demonstrated superior predictive performance across short-, medium-, and long-term forecast horizons compared to state-of-the-art models in the antenna assembly line dataset. These results were largely replicated in the complementary sink manufacturing dataset. This was achieved by integrating temporal feature extraction techniques such as patching and seasonal decomposition through MA and DFT methods, effectively capturing both short-range and long-range dependencies within the data. The results confirm that models incorporating specialized temporal feature extraction mechanisms significantly outperform those without them, particularly in handling the inherent noise and nonlinear dynamics of industrial time-series data.

Future work will explore the extension of these models to other industrial processes, the inclusion of additional external factors affecting OEE, and further refinement of model architectures to reduce computational complexity, while maintaining or improving accuracy.

Acknowledgment

The opinions expressed in this article are those of the authors and do not necessarily reflect the views of the European Commission.

References

[1] A. Evangelidis, N. Dimitriou, P. Charalampous, T. D. Mastos, and D. Tzovaras, "Efficient deep Q-learning for industrial equipment calibration in elevator manufacturing," *IEEE Trans. Ind. Informat.*, vol. 20, no. 10, pp. 12220–12230, Oct. 2024.

[2] J. Huang et al., "Deep reinforcement learning-based dynamic reconfiguration planning for digital twin-driven smart manufacturing systems with reconfigurable machine tools," *IEEE Trans. Ind. Informat.*, vol. 20, no. 11, pp. 13135–13146, Nov. 2024.

[3] L. Leontaris et al., "A blockchain-enabled deep residual architecture for accountable, in-situ quality control in Industry 4.0 with minimal latency," *Comput. Ind.*, vol. 149, Aug. 2023, Art. no. 103919. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0166361523000696

[4] A. Evangelidis, N. Dimitriou, L. Leontaris, D. Ioannidis, G. Tinker, and D. Tzovaras, "A deep regression framework toward laboratory accuracy in the shop floor of microelectronics," *IEEE Trans. Ind. Informat.*, vol. 19, no. 3, pp. 2652–2661, Mar. 2023.

[5] G. Margetis, K. C. Apostolakis, N. Dimitriou, D. Tzovaras, and C. Stephanidis, "Aligning emerging technologies onto I4.0 principles: Towards a novel architecture for zero-defect manufacturing," in *Proc. IEEE 27th Int. Conf. Emerg. Technol. Factory Autom. (ETFA)*, Sep. 2022, pp. 1–8.

[6] P. Krishnamurthy, R. Karri, and F. Khorrami, "Anomaly detection in real-time multi-threaded processes using hardware performance counters," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 666–680, 2020.

[7] P. Qiu, W. Li, and J. Li, "A new process control chart for monitoring short-range serially correlated data," *Technometrics*, vol. 62, no. 1, pp. 71–83, Jan. 2020.

[8] F. Psarommatis, G. May, and D. Kiritsis, "Predictive maintenance key control parameters for achieving efficient zero defect manufacturing," *Proc. CIRP*, vol. 104, pp. 80–84, Jan. 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2212827121009124

[9] F. Psarommatis, G. May, P.-A. Dreyfus, and D. Kiritsis, "Zero defect manufacturing: State-of-the-art review, shortcomings and future directions in research," *Int. J. Prod. Res.*, vol. 58, no. 1, pp. 1–17, Jan. 2020, doi: 10.1080/00207543.2019.1605228.

[10] F. Psarommatis, G. May, and V. Azamfirei, "Zero defect manufacturing in 2024: A holistic literature review for bridging the gaps and forward outlook," *Int. J. Prod. Res.*, vol. 62, pp. 1–37, Aug. 2024, doi: 10.1080/00207543.2024.2388217.

[11] F. Psarommatis and V. Azamfirei, "Zero defect manufacturing: A complete guide for advanced and sustainable quality management," *J. Manuf. Syst.*, vol. 77, pp. 764–779, Dec. 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0278612524002498

[12] F. Psarommatis, G. May, and V. Azamfirei, "Envisioning maintenance 5.0: Insights from a systematic literature review of Industry 4.0 and a proposed framework," *J. Manuf. Syst.*, vol. 68, pp. 376–399, Jun. 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0278612523000730

[13] P. Muchiri and L. Pintelon, "Performance measurement using overall equipment effectiveness (OEE): Literature review and practical application discussion," *Int. J. Prod. Res.*, vol. 46, no. 13, pp. 3517–3535, Jul. 2008, doi: 10.1080/00207540601142645.

[14] S. L. Ho and M. Xie, "The use of ARIMA models for reliability forecasting and analysis," *Comput. Ind. Eng.*, vol. 35, nos. 1–2, pp. 213–216, Oct. 1998. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0360835298000667

[15] E. S. Gardner Jr., "Exponential smoothing: The state of the art—Part II," *Int. J. Forecasting*, vol. 22, no. 4, pp. 637–666, Oct. 2006. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0169207006000392

[16] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" 2022, *arXiv:2205.13504*.

[17] S. Wang et al., "TimeMixer: Decomposable multiscale mixing for time series forecasting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, May 2024, pp. 1–27.

[18] J. C. Chen, T.-L. Chen, W.-J. Liu, C. C. Cheng, and M.-G. Li, "Combining empirical mode decomposition and deep recurrent neural networks for predictive maintenance of lithium-ion battery," *Adv. Eng. Informat.*, vol. 50, Oct. 2021, Art. no. 101405. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1474034621001579

[19] A. Sagheer and M. Kotb, "Time series forecasting of petroleum production using deep LSTM recurrent networks," *Neurocomputing*, vol. 323, pp. 203–213, Jan. 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231218311639

[20] C. Chen, Z. H. Zhu, J. Shi, N. Lu, and B. Jiang, "Dynamic predictive maintenance scheduling using deep learning ensemble for system health prognostics," *IEEE Sensors J.*, vol. 21, no. 23, pp. 26878–26891, Dec. 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9568926

[21] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. 30th Int. Conf. Int. Conf. Mach. Learn.*, vol. 28, Jan. 2012, pp. 1310–1318.

[22] Z. Geng, Z. Chen, Q. Meng, and Y. Han, "Novel transformer based on gated convolutional neural network for dynamic soft sensor modeling of industrial processes," *IEEE Trans. Ind. Informat.*, vol. 18, no. 3, pp. 1521–1529, Mar. 2022.

[23] S. Lee, J. Choi, and M. Su Kim, "Generative pre-training of time-series data for unsupervised fault detection in semiconductor manufacturing," 2023, *arXiv:2309.11427*.

[24] O. Ogunfowora and H. Najjaran, "A transformer-based framework for multi-variate time series: A remaining useful life prediction use case," 2023, *arXiv:2308.09884*.

[25] Y. Liu et al., "ITransformer: Inverted transformers are effective for time series forecasting," 2023, *arXiv:2310.06625*.

[26] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," 2018, *arXiv:1803.01271*.

[27] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," 2022, *arXiv:2211.14730*.

[28] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," 2022, *arXiv:2201.12740*.

[29] B. N. Oreshkin, D. Carpov, N. Chapados, and Y. Bengio, "N-BEATS: Neural basis expansion analysis for interpretable time series forecasting," 2019, *arXiv:1905.10437*.

[30] C. Challu, K. G. Olivares, B. N. Oreshkin, F. Garza, M. Mergenthaler-Canseco, and A. Dubrawski, "N-HiTS: Neural hierarchical interpolation for time series forecasting," 2022, *arXiv:2201.12886*.

[31] K. Yi et al., "Frequency-domain MLPs are more effective learners in time series forecasting," 2023, *arXiv:2311.06184*.

[32] Y. Liu, C. Li, J. Wang, and M. Long, "Koopa: Learning non-stationary time series dynamics with Koopman predictors," 2023, *arXiv:2305.18803*.

[33] T. Zhang et al., "Less is more: Fast multivariate time series forecasting with light sampling-oriented MLP structures," 2022, *arXiv:2207.01186*.

[34] A. Sherstinsky, "Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network," *Phys. D, Nonlinear Phenomena*, vol. 404, Mar. 2020, Art. no. 132306. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167278919305974

[35] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[36] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," 2014, *arXiv:1406.1078*.

[37] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997. [Online]. Available: https://ieeexplore.ieee.org/document/650093

[38] H. Jaeger, "The 'echo state' approach to analysing and training recurrent neural networks," German Nat. Res. Center Inf. Technol. GMD, Sankt Augustin, Germany, Tech. Rep., 148, 2001.

[39] J. Bradbury, S. Merity, C. Xiong, and R. Socher, "Quasi-recurrent neural networks," 2016, *arXiv:1611.01576*.

[40] M. Beck et al., "XLSTM: Extended long short-term memory," 2024, *arXiv:2405.04517*.

[41] A. R. Voelker, I. Kajić, and C. Eliasmith, "Legendre memory units: Continuous-time representation in recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32. Red Hook, NY, USA: Curran Associates, Sep. 2019, pp. 15544–15553. [Online]. Available: https://papers.nips.cc/paper/2019/hash/952285b9b7e7a1be5aa7849f32ffff05-Abstract.html

[42] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, "HiPPO: Recurrent memory with optimal polynomial projections," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33. Curran Associates, 2020, pp. 1474–1487. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/102f0bb6efb3a6128a3c750dd16729be-Abstract.html

[43] T. Zhou et al., "FiLM: Frequency improved legendre memory model for long-term time series forecasting," 2022, *arXiv:2205.08897*.

[44] R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, Mar. 1960, doi: 10.1115/1.3662552.

[45] A. Gu et al., "Combining recurrent, convolutional, and continuous-time models with linear state-space layers," 2021, *arXiv:2110.13985*.

[46] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," 2021, *arXiv:2111.00396*.

[47] J. T. H. Smith, A. Warrington, and S. W. Linderman, "Simplified state space layers for sequence modeling," in *Proc. 11th Int. Conf. Learn. Represent.*, Jan. 2022, pp. 1–35. [Online]. Available: https://openreview.net/forum?id=Ai8Hw3AXqks

[48] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.

[49] A. Vaswani et al., "Attention is all you need," 2017, *arXiv:1706.03762*.

[50] S. Li et al., "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," 2019, *arXiv:1907.00235*.

[51] N. Kitaev, Ł. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," 2020, *arXiv:2001.04451*.

[52] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," 2020, *arXiv:2012.07436*.

[53] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," 2021, *arXiv:2106.13008*.

[54] G. Woo, C. Liu, D. Sahoo, A. Kumar, and S. Hoi, "ETSformer: Exponential smoothing transformers for time-series forecasting," 2022, *arXiv:2202.01381*.

[55] Y. Zhang and J. Yan, "CrossFormer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 1–21.

[56] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, "Temporal convolutional networks for action segmentation and detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1003–1012. [Online]. Available: https://ieeexplore.ieee.org/document/8099596

[57] H. Wang, J. Peng, F. Huang, J. Wang, J. Chen, and Y. Xiao, "MICN: Multi-scale local and global context modeling for long-term series forecasting," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 1–22.

[58] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2D-variation modeling for general time series analysis," 2022, *arXiv:2210.02186*.

[59] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, vol. 16, Jun. 2016, pp. 770–778.

[60] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 5987–5995.

[61] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 11966–11976.

[62] Y. Tang et al., "A survey on machine learning models for financial time series forecasting," *Neurocomputing*, vol. 512, pp. 363–380, Nov. 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S092523122201089X

[63] Y. O. Ouma, R. Cheruyot, and A. N. Wachera, "Rainfall and runoff time-series trend analysis using LSTM recurrent neural network and wavelet neural network with satellite-based meteorological data: Case study of Nzoia hydrologic basin," *Complex Intell. Syst.*, vol. 8, no. 1, pp. 213–236, Feb. 2022, doi: 10.1007/s40747-021-00365-2.

[64] K. Ratna Prakarsha and G. Sharma, "Time series signal forecasting using artificial neural networks: An application on ECG signal," *Biomed. Signal Process. Control*, vol. 76, Jul. 2022, Art. no. 103705. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1746809422002270

[65] A. K. Dubey, A. Kumar, V. García-Díaz, A. K. Sharma, and K. Kanhaiya, "Study and analysis of SARIMA and LSTM in forecasting time series data," *Sustain. Energy Technol. Assessments*, vol. 47, Jul. 2021, Art. no. 101474. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2213138821004847

[66] W. Li and K. L. E. Law, "Deep learning models for time series forecasting: A review," *IEEE Access*, vol. 12, pp. 92306–92327, 2024.

[67] Z. Chen, M. Ma, T. Li, H. Wang, and C. Li, "Long sequence time-series forecasting with deep learning: A survey," *Inf. Fusion*, vol. 97, Sep. 2023, Art. no. 101819. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1566253523001355
[68] H. Hewamalage, C. Bergmeir, and K. Bandara, "Global models for time series forecasting: A simulation study," *Pattern Recognit.*, vol. 124, Apr. 2022, Art. no. 108441. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0031320321006178
[69] P.-A. Dreyfus, F. Psarommatis, G. May, and D. Kiritsis, "Virtual metrology as an approach for product quality estimation in Industry 4.0: A systematic review and integrative conceptual framework," *Int. J. Prod. Res.*, vol. 60, no. 2, pp. 742–765, Jan. 2022, doi: 10.1080/00207543.2021.1976433.
[70] H. Gao, Y. Li, Y. Zhao, and Y. Song, "Dual channel feature attention-based approach for RUL prediction considering the spatiotemporal difference of multisensor data," *IEEE Sensors J.*, vol. 23, no. 8, pp. 8514–8525, Apr. 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10063980
[71] A. Essien and C. Giannetti, "A deep learning model for smart manufacturing using convolutional LSTM neural network autoencoders," *IEEE Trans. Ind. Informat.*, vol. 16, no. 9, pp. 6069–6078, Sep. 2020. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8967003
[72] X. Li, F. Bi, X. Yang, and X. Bi, "An echo state network with improved topology for time series prediction," *IEEE Sensors J.*, vol. 22, no. 6, pp. 5869–5878, Mar. 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9701926
[73] K. Zheng, B. Qian, S. Li, Y. Xiao, W. Zhuang, and Q. Ma, "Long-short term echo state network for time series prediction," *IEEE Access*, vol. 8, pp. 91961–91974, 2020.
[74] C.-Y. Hsu and W.-C. Liu, "Multiple time-series convolutional neural network for fault detection and diagnosis and empirical study in semiconductor manufacturing," *J. Intell. Manuf.*, vol. 32, no. 3, pp. 823–836, Mar. 2021, doi: 10.1007/s10845-020-01591-0.
[75] S. Nouhi and M. Pour, "Prediction of surface roughness of various machining processes by a hybrid algorithm including time series analysis, wavelet transform and multi view embedding," *Measurement*, vol. 184, Jul. 2021, Art. no. 109904. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0263224121008435
[76] A. Nikseresht, M. Zandieh, and M. Shokouhifar, "Time series forecasting using improved empirical Fourier decomposition and high-order intuitionistic FCM: Applications in smart manufacturing systems," *IEEE Trans. Fuzzy Syst.*, early access, Sep. 16, 2024, doi: 10.1109/TFUZZ.2024.3462631.
[77] X. Yuan, L. Li, Y. Shardt, Y. Wang, and C. Yang, "Deep learning with spatiotemporal attention-based LSTM for industrial soft sensor model development," *IEEE Trans. Ind. Electron.*, vol. 68, no. 5, pp. 4404–4414, May 2021. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9062588
[78] C. I. Valero, F. Boronat, M. Esteve, and C. E. Palau, "AI for detecting variations in the OEE data reception rate in the manufacturing industry," in *Proc. 10th ECCOMAS Thematic Conf. Smart Struct. Mater.*, 2023, pp. 1185–1195. [Online]. Available: https://api.semanticscholar.org/CorpusID
[79] J. Ni, W. C. Tang, and Y. Xing, "Assembly process optimization for reducing the dimensional error of antenna assembly with abundant rivets," *J. Intell. Manuf.*, vol. 29, no. 1, pp. 245–258, Jan. 2018, doi: 10.1007/s10845-015-1105-x.
[80] K. Choi, J. Yi, C. Park, and S. Yoon, "Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines," *IEEE Access*, vol. 9, pp. 120043–120065, 2021. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9523565
[81] P. Lara-Benítez, M. Carranza-García, and J. C. Riquelme, "An experimental review on deep learning architectures for time series forecasting," *Int. J. Neural Syst.*, vol. 31, no. 3, Mar. 2021, Art. no. 2130001, doi: 10.1142/S0129065721300011.
[82] S. Deng, O. Sprangers, M. Li, S. Schelter, and M. de Rijke, "Domain generalization in time series forecasting," *ACM Trans. Knowl. Discovery Data*, vol. 18, no. 5, pp. 1–24, Feb. 2024, doi: 10.1145/3643035.
[83] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, "Reversible instance normalization for accurate time-series forecasting against distribution shift," in *Proc. Int. Conf. Learn. Represent.*, 2022, pp. 1–25. [Online]. Available: https://openreview.net/forum?id=cGDAkQo1C0p
[84] O. Blaskowitz and H. Herwartz, "On economic evaluation of directional forecasts," *Int. J. Forecasting*, vol. 27, no. 4, pp. 1058–1065, Oct. 2011. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0169207011000057
[85] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2D-variation modeling for general time series analysis," in *Proc. Int. Conf. Learn. Represent.*, 2022.
[86] A. Das, W. Kong, A. Leach, S. Mathur, R. Sen, and R. Yu, "Long-term forecasting with TiDE: Time-series dense encoder," 2023, *arXiv:2304.08424*.
[87] L. Ke, G. Hu, Y. Yang, and Y. Liu, "Fault diagnosis for modular multilevel converter switching devices via multimodal attention fusion," *IEEE Access*, vol. 11, pp. 135035–135048, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10328833
[88] Y. Zhang et al., "A late fusion CNN for digital matting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 7461–7470. [Online]. Available: https://openaccess.thecvf.com/content_CVPR_2019/html/Zhang_A_Late_Fusion_CNN_for_Digital_Matting_CVPR_2019_paper.html
[89] H. Wang, J. Zhou, H. Chen, B. Xu, and Z. Shen, "Hydraulic system fault diagnosis decoupling method based on 2D time-series modeling and self-attention fusion," *Sci. Rep.*, vol. 14, no. 1, p. 15620, Jul. 2024. [Online]. Available: https://www.nature.com/articles/s41598-024-66541-9
[90] Y. Wang et al., "TimeXer: Empowering transformers for time series forecasting with exogenous variables," 2024, *arXiv:2402.19072*.

Dimitrios Oikonomou received the bachelor's degree in applied informatics from the University of Macedonia, Thessaloniki, Greece, in 2022.

He has been a Research Assistant with the Information Technologies Institute, Centre for Research and Technology Hellas, Thessaloniki, since March 2023. His research interests include machine learning and time-series analysis with a focus industrial applications.



Lampros Leontaris (Member, IEEE) received the Diploma degree in electrical and computer engineering from the Aristotle University of Thessaloniki, Thessaloniki, Greece, in 2016.

He has been a Research Assistant with the Information Technologies Institute, Centre for Research and Technology Hellas, Thessaloniki, since March 2017. His research interests include computational intelligence methods, machine learning, and fuzzy systems.



Nikolaos Dimitriou (Member, IEEE) received the Diploma and Ph.D. degrees in electrical and computer engineering from the Aristotle University of Thessaloniki, Greece, in 2007 and 2014, respectively.

He is currently a Senior Researcher (Grade C') with the Information Technologies Institute, Centre for Research and Technology Hellas (CERTH), Thessaloniki. His research interests include computer vision and deep learning, with a particular focus on industrial applications.



Dimitrios Tzovaras (Senior Member, IEEE) received the Diploma and Ph.D. degrees in electrical and computer engineering from the Aristotle University of Thessaloniki, Thessaloniki, Greece, in 1992 and 1997, respectively.

He is currently a Senior Researcher (Grade A') and the President of the Board with the Centre for Research and Technology Hellas, Thessaloniki. His main research interests include visual analytics, 3-D object recognition, search and retrieval, behavioral biometrics, assistive technologies, information and knowledge management, computer graphics, and virtual reality.